SETUP & OPERATIONS GUIDE · FREE EDITION

AI Influencer Bot on $0 infrastructure

A complete, from-scratch manual for running an automated Instagram news-carousel bot with human approval in Telegram — using only free services: GitHub Actions, a Cloudflare Worker, Google Gemini and Cloudflare Workers AI.

What you will have at the end

- A draft post (images + caption) delivered to your Telegram every day
- Two buttons — **Approve & Publish** and **Reject / Edit** — that really control what goes live on Instagram
- Nothing running 24/7, nothing to pay for, nothing to keep patched
- A clear routine for renewing keys and keeping the bot healthy

Contents

1 · How it works (architecture & theory) 3

2 · Before you start (accounts, values, time) 7

3 · Step-by-step setup 8

Step 1 – GitHub repository 8

Step 2 – Allow workflows to write 10

Step 3 – Telegram bot & chat ID 11

Step 4 – Gemini API key 12

Step 5 – Cloudflare account & Workers AI token 13

Step 6 – Instagram Graph API token & account ID 14

Step 7 – Store secrets in GitHub 16

Step 8 – GitHub token for the relay 17

Step 9 – Deploy the Cloudflare Worker 18

Step 10 – Connect Telegram to the Worker 20

Step 11 – First run & end-to-end test 21

4 · Daily use & customisation 23

5 · Keeping it running: keys, security & maintenance 25

6 · Troubleshooting 28

Appendix · Cheat-sheet & Worker code 29

ABOUT THE SCREENSHOTS

Real screenshots in this guide were taken from a working installation and anonymised: account names, e-mail addresses and avatars were replaced by `user123`. Pages that display real secrets (API keys, tokens, the BotFather chat) are **never** shown as screenshots — those steps use clearly labelled illustrations instead. When you follow the guide, never share screenshots of those pages either.

THIRD-PARTY WEBSITES CHANGE

Google, Meta, Cloudflare and GitHub redesign their dashboards regularly. Button names and positions in this guide may differ slightly from what you see; the *concepts* and the names of the values you need stay the same. Where a step depends on a fast-changing external flow (especially Instagram/Meta), the official documentation is linked in the text.

1 · How it works

1.1 What the bot does

Every day the bot reads a list of AI/tech news feeds (RSS), picks a fresh article, asks **Google Gemini** to write an Instagram caption and the text for several carousel slides, asks **Cloudflare Workers AI (FLUX)** to paint matching background pictures, composes the final slides with **Pillow** (a Python imaging library — brand colours, fonts, headline, handle), and sends the finished draft to **you in Telegram**. Nothing is published until you tap **Approve**. If you tap **Reject / Edit**, you write feedback in plain language ("shorter, more critical tone, fewer hashtags") and the bot regenerates the draft. This pattern — automation that prepares work and a person who authorises it — is called **human-in-the-loop**.

1.2 The core design problem: no always-on server

The original version of this bot was a small web server (FastAPI + a scheduler) hosted on a paid platform. A server is needed for two reasons: (1) something must *wake up at a set time*, and (2) something must *listen* for the moment you tap a Telegram button. The free edition removes the server by splitting these two jobs among free event-driven services:

- **Time-triggered work → GitHub Actions.** A workflow with a `cron` schedule starts a short-lived virtual machine, runs the Python code, and shuts it down. Public repositories get free Actions minutes.
- **Listening → Cloudflare Worker (a "relay").** Telegram can be told to deliver every update to a **webhook** — an HTTPS address. A Cloudflare Worker is a tiny piece of JavaScript running at that address; it is "always on" from the outside but costs nothing when idle. It immediately acknowledges the tap and forwards the update to GitHub.
- **Event-triggered work → GitHub `repository_dispatch`.** The Worker calls the GitHub API with an event named `telegram_update`. This starts a second workflow that runs the handler once, answers you in Telegram and exits.

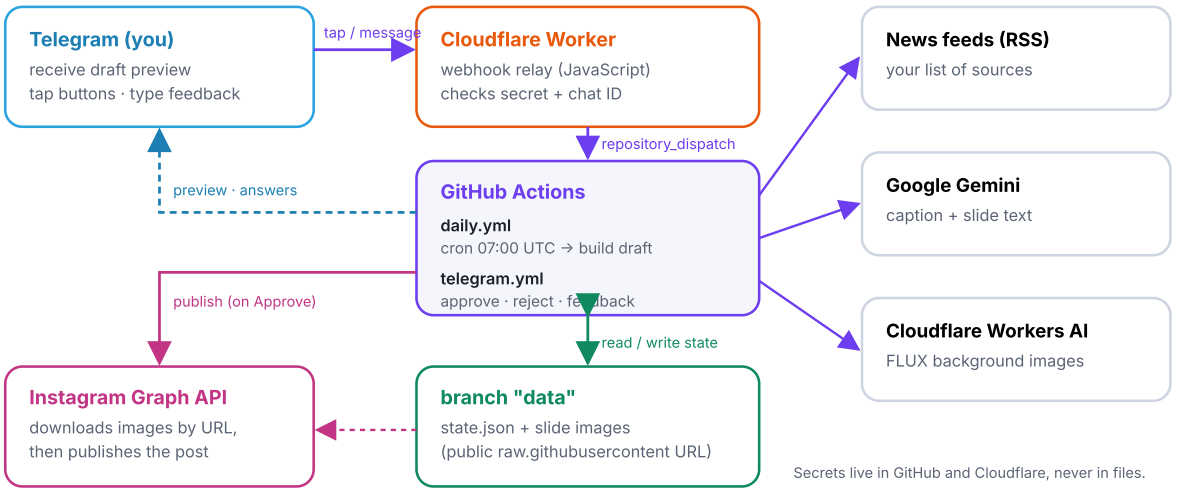


Figure 1. Components of the free edition. Taps travel Telegram → Worker → GitHub; GitHub Actions does the real work, keeps state on the `data` branch and talks to Telegram and Instagram.

1.3 The three life-cycles

A · The daily draft (`daily.yml`)

1. GitHub starts the workflow at 07:00 UTC (it may start a few minutes late — GitHub's scheduler is "best effort").
2. The job restores state by checking out the `data` branch into a local `data/` folder (`scripts/data_branch.sh pull`).
3. The code scrapes the feeds, skips articles already used, and calls Gemini for text and FLUX for images; Pillow renders 1080×1350 slides.
4. Images and `state.json` are pushed back to the `data` branch **before** the preview is sent — so by the time you can tap Approve, the images are already publicly reachable.
5. The bot sends the preview and keyboard to your Telegram chat, and the job ends.

B · A button tap or message (`telegram.yml`)

1. Telegram POSTs the update to your Worker URL with the header `X-Telegram-Bot-Api-Secret-Token` .
2. The Worker checks the header against `WEBHOOK_SECRET` and the chat ID against `TELEGRAM_CHAT_ID` (everyone else is ignored), tells Telegram "Working on it..." so the button stops spinning, and fires a `repository_dispatch` event carrying the whole update as JSON.
3. GitHub starts `telegram.yml` ; the Python handler reads the update and acts: **Approve** → publish via the Instagram Graph API; **Reject** → ask for feedback; plain text → regenerate with that feedback; `/status` , `/generate` , `/cancel` → the matching command.
4. Total latency from tap to answer is typically 20–60 seconds, because a fresh runner has to boot.

C · Publishing to Instagram

Instagram's API does not accept file uploads. It accepts *URLs* and downloads the images itself. That is why the repository must be **public**: Instagram fetches `raw.githubusercontent.com/…/slide_1.jpg` . For a carousel the bot creates one child "container" per image, one parent carousel container, waits until Instagram reports it as finished, then calls `media_publish` . Once published, the draft's images are removed from the public branch.

1.4 Key concepts in one table

Concept	What it means here
GitHub Actions	GitHub's built-in automation: YAML files in `.github/workflows/` describe when to start a temporary Linux machine and which commands to run on it.
cron schedule	A time pattern (`0 7 * * *` = every day 07:00 UTC). Always UTC, never your local time.
Webhook	"Call this URL when something happens." Telegram calls your Worker each time you tap a button or send a message.
Cloudflare Worker	A small JavaScript function hosted on Cloudflare's edge network. The free plan allows 100,000 requests per day — far more than a single-user bot needs.

repository_dispatch	A GitHub API call that starts a workflow from outside, optionally with a JSON payload. It needs a token with write access to the repository contents.
Secret	An encrypted value stored by GitHub/Cloudflare. It cannot be read back after saving — only replaced — and is hidden in logs.
`data` branch	A Git branch used as a tiny database/file store. It is force-pushed as a single commit every time, so the repository never grows.
Concurrency group	A workflow setting that makes runs wait in line, so two jobs never edit `state.json` simultaneously.
Access token vs API key	Both are passwords for software. Treat all of them as secrets: anyone holding one can act as you on that service.

1.5 Free-tier limits (and what happens when you hit them)

Service	Free allowance (at the time of writing)	If exceeded
GitHub Actions	Unlimited minutes for public repositories (standard runners)	Not a concern for ~2–3 runs/day. A private repo would get 2,000 min/month instead.
Cloudflare Worker	100,000 requests/day	Requests would be rejected; practically unreachable.
Workers AI (FLUX)	10,000 "neurons" per day, resets 00:00 UTC	Image generation fails with an error message in Telegram. Retry after reset or use /generate tomorrow.
Gemini API	Free tier with per-minute and per-day limits that differ by model (see Google AI Studio → rate limits)	Error in Telegram; the bot can fall back to a second model if `GEMINI_FALLBACK_MODEL` is set.
Telegram Bot API	Free	—
Instagram Graph API	Free, but with a daily cap of API-published posts per account	Publishing error; see Meta's current content-publishing limit.

> **LIMITS CHANGE**
> Free-tier numbers are set by the providers and change over time. Treat the figures above as orientation and check each provider's pricing page before relying on them.

1.6 Security model — what is and is not public

- **Public:** the source code, the workflow files, the `data` branch with draft images (and `state.json` with captions and article links).
- **Private:** every key and token. They live only in GitHub Actions secrets and Cloudflare Worker secrets, never in files.

- **Access control:** the Worker only accepts requests carrying your secret header and only from your chat ID. A stranger who finds the bot cannot trigger publishing.
- **Blast radius:** the GitHub token used by the Worker is fine-grained — it is limited to one repository and one permission (Contents: read & write).
- **Trade-off:** drafts are publicly readable (by guessing a random draft URL) until published. Do not put anything confidential into a draft.

2 · Before you start

2.1 Accounts you need

Account	Used for	Needs a card?
GitHub	Code, scheduled runs, state storage	No
Telegram	Approval chat with your bot	No
Google (AI Studio)	Gemini API key	No (free tier)
Cloudflare	Worker relay + image generation	No
Instagram **Professional** account (Business or Creator) + a Facebook Page linked to it + Meta developer account	Publishing through the Graph API	No

INSTAGRAM IS THE LONGEST STEP

The Instagram Graph API only works for **Professional** accounts. If your account is personal, switch it in Instagram → Settings → Account type, and connect it to a Facebook Page. Allow extra time for Meta's setup screens (Step 6).

2.2 The nine values you will collect

Keep a private notes file (a password manager is ideal — **not** a screenshot, not a chat, not the repository). Fill it in as you go; each step below tells you which value you just obtained.

#	Value	Where it comes from	Ends up in
1	`TELEGRAM_BOT_TOKEN` SECRET	@BotFather (Step 3)	GitHub secrets + Worker
2	`TELEGRAM_CHAT_ID`	Your numeric Telegram ID (Step 3)	GitHub secrets + Worker
3	`GEMINI_API_KEY` SECRET	Google AI Studio (Step 4)	GitHub secrets
4	`CLOUDFLARE_ACCOUNT_ID`	Cloudflare dashboard (Step 5)	GitHub secrets
5	`CLOUDFLARE_API_TOKEN` SECRET	Cloudflare → API Tokens (Step 5)	GitHub secrets
6	`INSTAGRAM_ACCESS_TOKEN` SECRET	Meta Graph API Explorer (Step 6)	GitHub secrets
7	`INSTAGRAM_ACCOUNT_ID`	Graph API call (Step 6)	GitHub secrets
8	`GH_TOKEN` SECRET	GitHub fine-grained token (Step 8)	Worker only

9	`WEBHOOK_SECRET` SECRET	You invent it (Step 9): letters, digits, `_`, `-` only	Worker + setWebhook call

2.3 Time and tools

Plan 60–90 minutes the first time (most of it Instagram/Meta). You need only a web browser and, for two commands in Step 10, a terminal: **PowerShell** on Windows (use `curl.exe`, not `curl`), Terminal on macOS/Linux.

> **GOLDEN RULE FOR THE WHOLE SETUP**
> Wherever this guide shows `<something>` in angle brackets, replace the **whole** thing including the brackets with your value. Typing `bot<TOKEN>` is the most common beginner mistake — it must become `bot123456:ABC…`.

3 · Step-by-step setup

1 Create the GitHub repository

≈ 10 min
no secrets involved

1. Sign in at **github.com** (or create a free account). Click **+ → New repository**.
2. Repository name: `ig-bot` (any name works). Visibility: **Public** — required so Instagram can download the images and so Actions are free. Leave "Add README", ".gitignore" and "license" off.
3. Click **Create repository**.

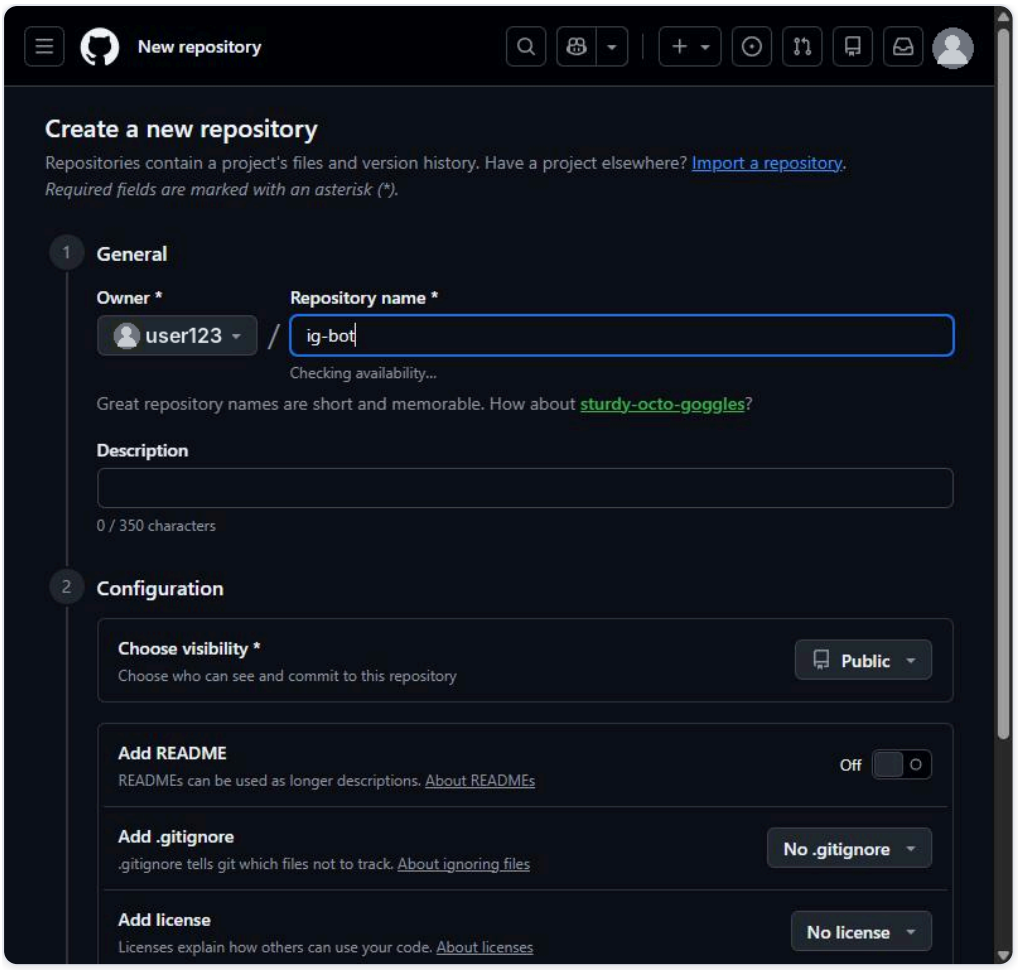


Figure 2. New repository form: owner, name `ig-bot`, visibility **Public**.

4. On the empty repository page choose **uploading an existing file**. Drag the **contents** of the `free-version` folder into the page (folders `assets`, `scripts`, `services`, `worker` and the files `config.py`, `run_daily.py`, `handle_update.py`, `requirements.txt`, `.env.example`, `.gitignore`). Commit with the default message.
5. The two workflow files must live in a folder named `.github/workflows/`. Upload `daily.yml` and `telegram.yml` from the `github-workflows` folder in a separate upload: use **Add file → Upload files**, and in the address bar change the URL so it ends with `/upload/main/.github/workflows`, then drop the two files in. (Alternatively use **Add file → Create new file**, type `.github/workflows/daily.yml` as the name — the slashes create folders — and paste the content.)

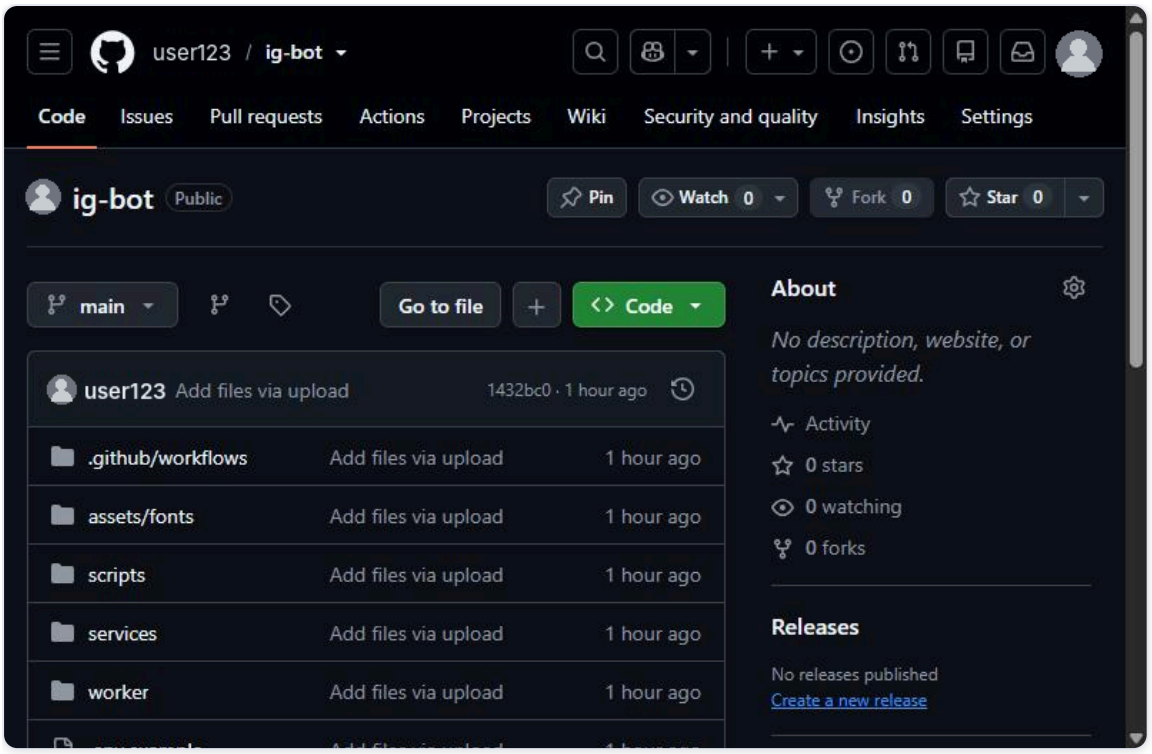


Figure 3. What the repository should look like afterwards: `.github/workflows`, `assets/fonts`, `scripts`, `services`, `worker` and the root files.

> **CHECK**
> Open `.github/workflows/` in the repository: you must see exactly `daily.yml` and `telegram.yml`. If the folder is missing, the bot will never run.

2 Allow workflows to write to the repository

≈ 1 min

The bot stores its state by pushing to the `data` branch, so the built-in workflow token needs write access.

1. Repository → **Settings → Actions → General**.
2. Scroll to **Workflow permissions**, choose **Read and write permissions**, click **Save**.

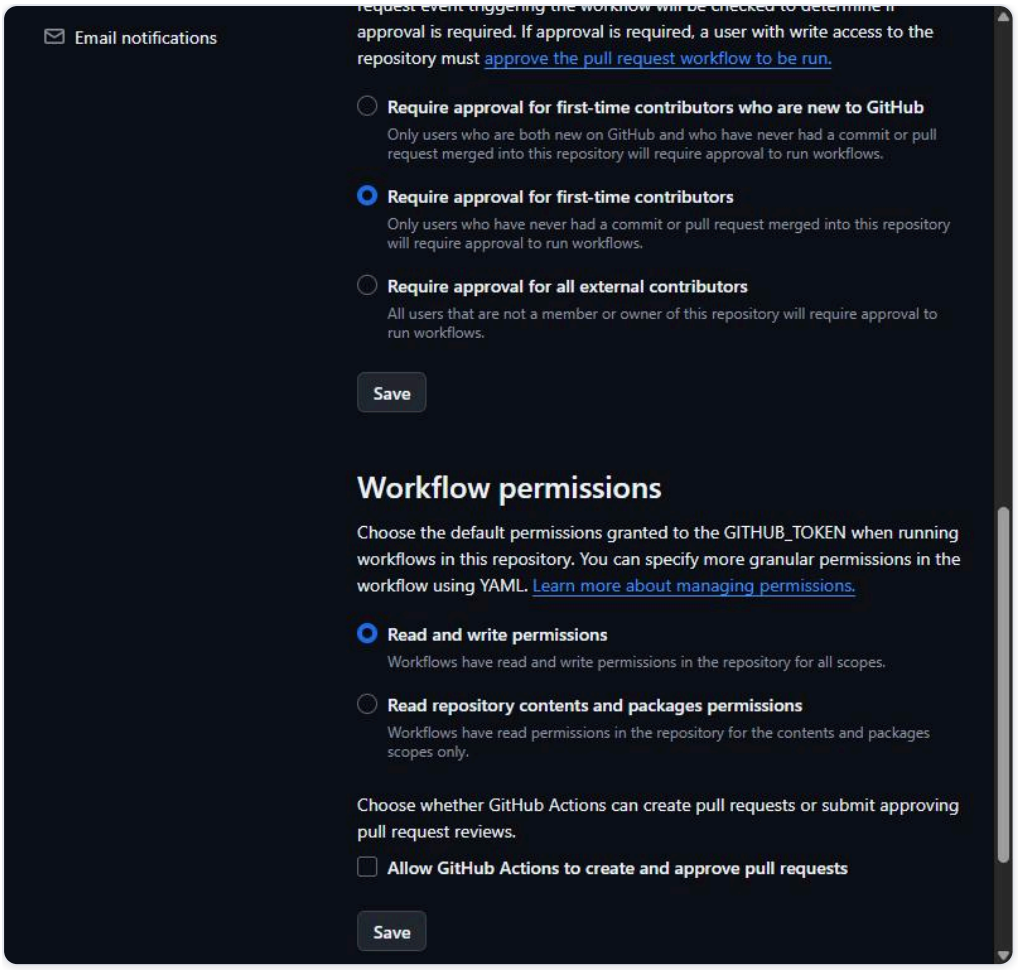


Figure 4. Workflow permissions set to **Read and write**.

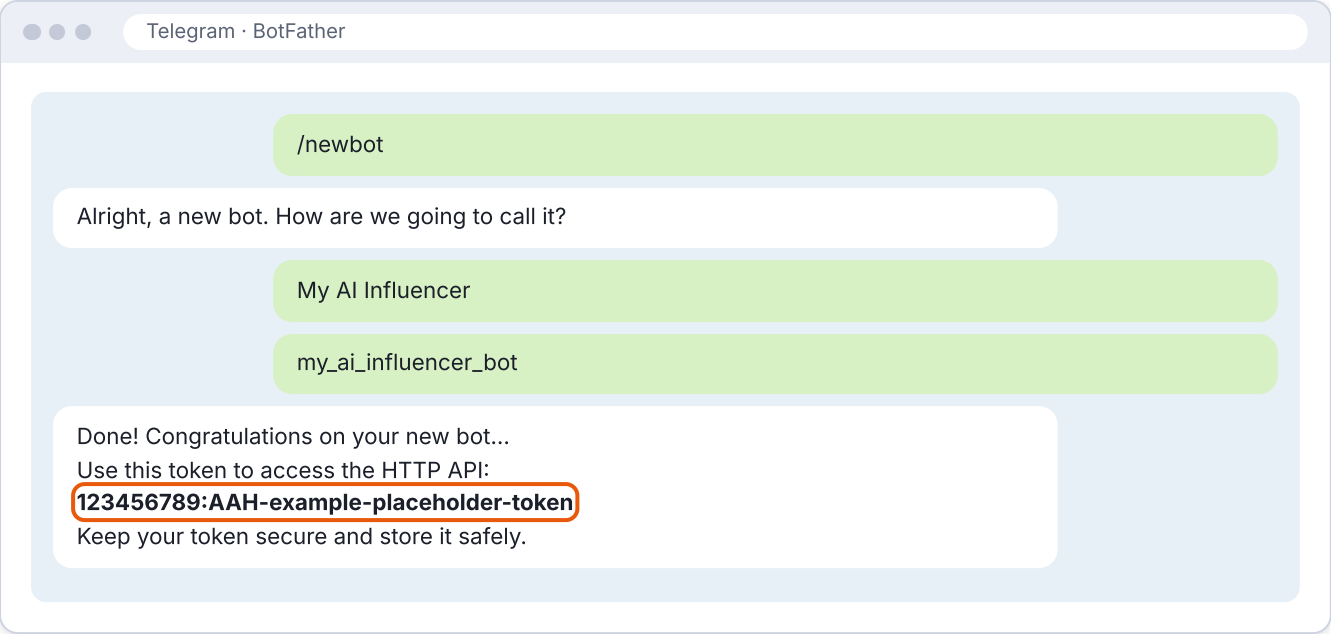
3 Telegram bot and your chat ID

≈ 5 min
yields values #1, #2

> **SENSITIVE — DO NOT SCREENSHOT OR SHARE**
> The bot token shown by BotFather is a password. Anyone who has it can control your bot. This guide therefore uses an illustration, not a real screenshot.

1. In Telegram search for **@BotFather** (the official one has a blue check mark) and open it.
2. Send `/newbot`. Choose a display name (e.g. "My AI Influencer") and then a unique username that ends in `bot` (e.g. `my_ai_influencer_bot`).
3. BotFather answers with a line "Use this token to access the HTTP API:" followed by a token of the form `123456789:AAH…`. Copy it into your private notes as `TELEGRAM_BOT_TOKEN`.
4. Open your new bot in Telegram and press **Start** / send any message ("hello"). This is required so the bot is allowed to message you.

Illustration — not a real screenshot; the token is an invented placeholder



Find your chat ID

The bot only obeys one person — you — identified by a numeric ID. Two easy ways:

- **Option A (easiest):** message a bot such as **@userinfobot**; it replies with your numeric `Id`.
- **Option B:** after sending "hello" to your own bot, open `https://api.telegram.org/bot<TOKEN>/getUpdates` in a browser (with your token substituted) and read `"chat":{"id": …}`.

DO THIS BEFORE STEP 10

Option B stops working once a webhook is set (Telegram then returns an error because updates are delivered to the Worker instead). Get your chat ID first. Save it as `TELEGRAM_CHAT_ID`. It is only digits (it can be negative for group chats; for a private chat it is positive).

4 Gemini API key (text generation)

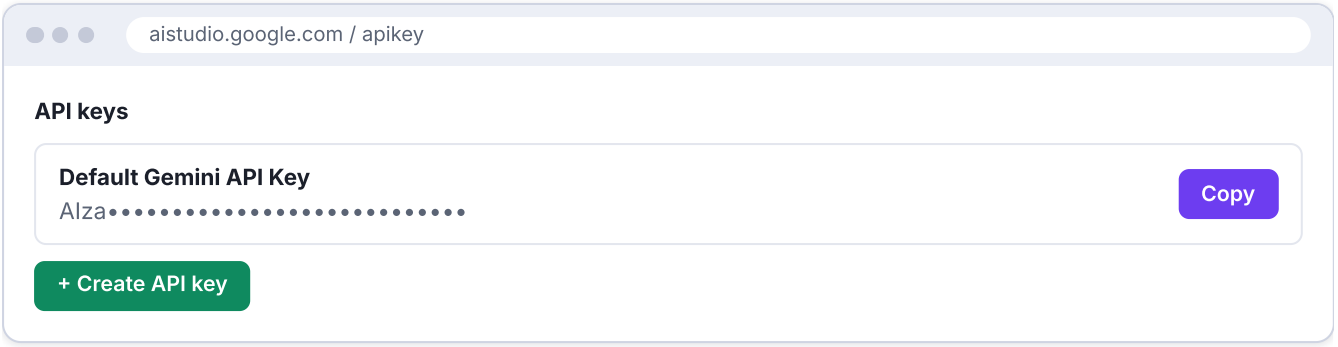
≈ 3 min
yields value #3

SENSITIVE PAGE — ILLUSTRATION ONLY

The API-keys page lists your real keys. Never screenshot it.

1. Open **aistudio.google.com** and sign in with a Google account.
2. Go to **Get API key** → **Create API key**. If asked, create it in a new project.
3. Copy the key (starts with `AIza…`) and save it as `GEMINI_API_KEY`.

Illustration — not a real screenshot



FREE TIER & PRIVACY

On Google's free tier the content you send can be used to improve their products. That is fine for public news captions, but do not send anything private through this key. The rate limits shown in AI Studio decide how many drafts per day you can generate; one daily draft with a few regenerations is far below them.

5 Cloudflare account and Workers AI token

≈ 8 min
yields values #4, #5

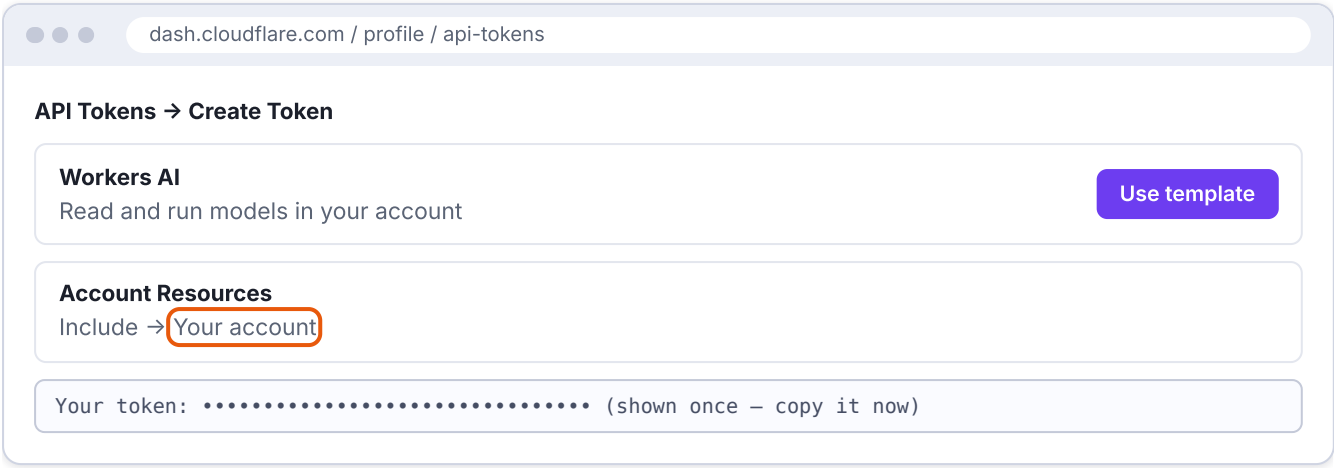
Cloudflare does two jobs for you: it paints the background images (Workers AI, FLUX model) and later hosts the Worker relay. Use one account for both.

1. Create a free account at **dash.cloudflare.com** (no card is needed for the Workers free plan) and verify your e-mail.
2. **Account ID:** open **Workers & Pages** (left menu). The Account ID is shown on the overview page (right-hand column) and is also the long hexadecimal part of the dashboard URL. Save it as `CLOUDFLARE_ACCOUNT_ID`. It is an identifier, not a password.
3. **API token:** click your profile icon → **My Profile → API Tokens → Create Token**. Use the **Workers AI** template (or a custom token with permission *Account → Workers AI → Read/Edit*), scope it to your account, and create it.
4. Copy the token **immediately** — Cloudflare shows it only once. Save it as `CLOUDFLARE_API_TOKEN`.

SENSITIVE — ILLUSTRATION ONLY

The token-created screen displays the secret value. Do not capture it.

Illustration — not a real screenshot



LEAST PRIVILEGE

Give this token only the Workers AI permission. If it ever leaks, the worst case is someone burning your free image allowance.

6 Instagram: access token and account ID

≈ 30–45 min
yields values #6, #7

SENSITIVE — ILLUSTRATION ONLY

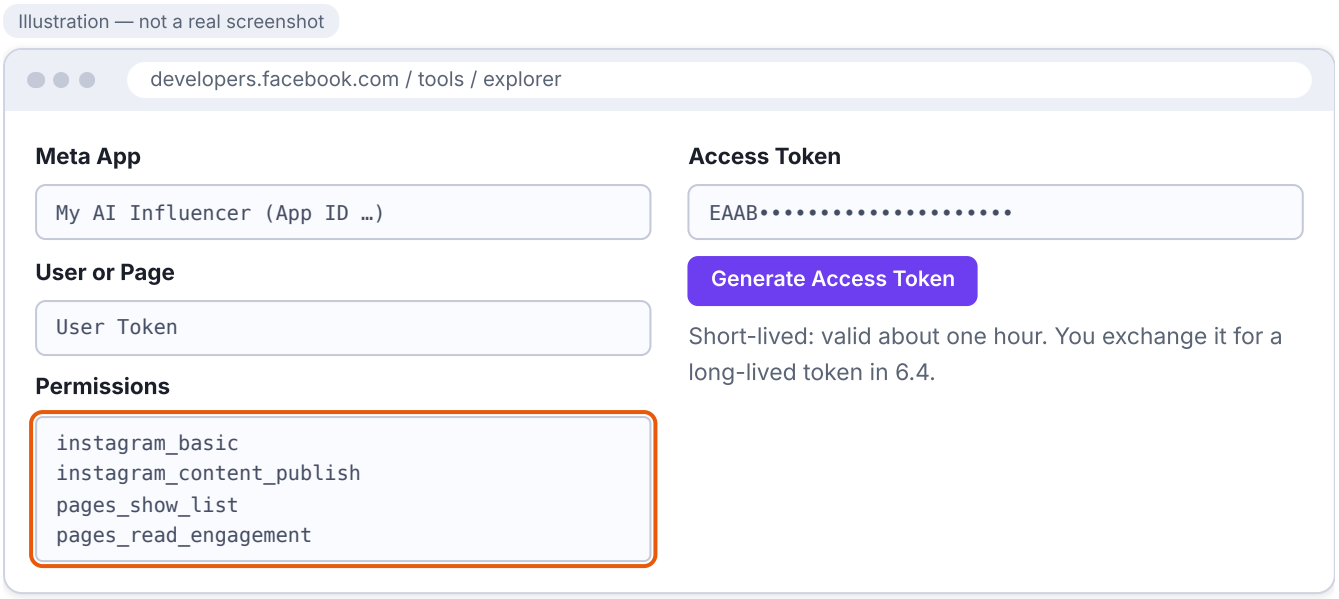
The Meta pages show your app secret and access tokens. Do not capture them.

6.1 Prerequisites

- An Instagram **Professional** account (Business or Creator).
- A **Facebook Page** that is linked to that Instagram account (Instagram → Settings → Account → Linked accounts / Page).
- A free **Meta for Developers** account (developers.facebook.com).

6.2 Create a Meta app

1. developers.facebook.com → **My Apps → Create App**. Choose the use case related to **Instagram / Content publishing** (names change; pick the one that mentions managing Instagram content) and a Business type if asked.
2. Note the **App ID** and **App Secret** (App settings → Basic). The secret is a password — treat it as a secret.
3. The app can stay in **Development mode**: while you are the app's administrator, you may publish to your own connected account.



6.3 Generate a user token in the Graph API Explorer

1. Open **Tools → Graph API Explorer**. Select your app and **User Token**.
2. Add the permissions `instagram_basic`, `instagram_content_publish`, `pages_show_list` and `pages_read_engagement` (the exact list the platform requires can change — see *developers.facebook.com/docs/instagram-platform*).
3. Click **Generate Access Token**, log in, and select the Facebook Page + Instagram account when Meta asks which assets the app may access.

6.4 Exchange it for a long-lived token (≈ 60 days)

Open this URL in a browser (replace the three placeholders, without brackets). It returns JSON with a longer-lived `access_token`:

```
https://graph.facebook.com/v22.0/oauth/access_token
  ?grant_type=fb_exchange_token
  &client_id=APP_ID
  &client_secret=APP_SECRET
  &fb_exchange_token=SHORT_LIVED_TOKEN
```

Save the returned value as `INSTAGRAM_ACCESS_TOKEN`. Note the date: it will expire in about 60 days (see Section 5).

> **THE URL CONTAINS YOUR APP SECRET**
> Browser history and screenshots can leak it. Prefer running it in the Graph API Explorer or a private window and clear the history afterwards. If you ever exposed the app secret, reset it in App settings → Basic.

6.5 Find the Instagram account ID

In the Graph API Explorer (using the long-lived token) run, in this order:

```
# 1) list your Pages → note the Page "id"
GET /me/accounts

# 2) ask that Page for its linked Instagram account
GET /PAGE_ID?fields=instagram_business_account
```

The `id` inside `instagram_business_account` (a long number) is `INSTAGRAM_ACCOUNT_ID`.

OPTIONAL: A NON-EXPIRING TOKEN

The response of `/me/accounts` (when called with a long-lived user token) includes a Page access token for each Page. Page tokens derived this way are documented by Meta as not expiring on a timer, which can spare you the 60-day renewal. If you use one, put it in `INSTAGRAM_ACCESS_TOKEN` instead and confirm in Meta's current documentation that it still has the permissions the bot needs. Test it with a draft before relying on it.

6.6 Quick token test (optional but recommended)

```
https://graph.facebook.com/v22.0/INSTAGRAM_ACCOUNT_ID?fields=username&access_token=YOUR_TOKEN
```

If the answer contains your Instagram username, the token and ID work together.

API VERSION

The bot calls Graph API **v22.0** by default (variable `INSTAGRAM_GRAPH_VERSION`). Meta retires old versions after a couple of years; if publishing suddenly fails with a version error, set that variable to a current version.

7 Store the secrets in GitHub

≈ 5 min

1. Repository → **Settings → Secrets and variables → Actions → New repository secret**.
2. Add the seven secrets one by one. The **name** must match exactly (capital letters, underscores); the **value** is pasted from your notes, with no spaces or line breaks at the ends.

Secret name	Required?
`GEMINI_API_KEY`	Yes
`TELEGRAM_BOT_TOKEN`	Yes
`TELEGRAM_CHAT_ID`	Yes
`INSTAGRAM_ACCESS_TOKEN`	Yes
`INSTAGRAM_ACCOUNT_ID`	Yes
`CLOUDFLARE_ACCOUNT_ID`	Recommended (free images)
`CLOUDFLARE_API_TOKEN`	Recommended (free images)

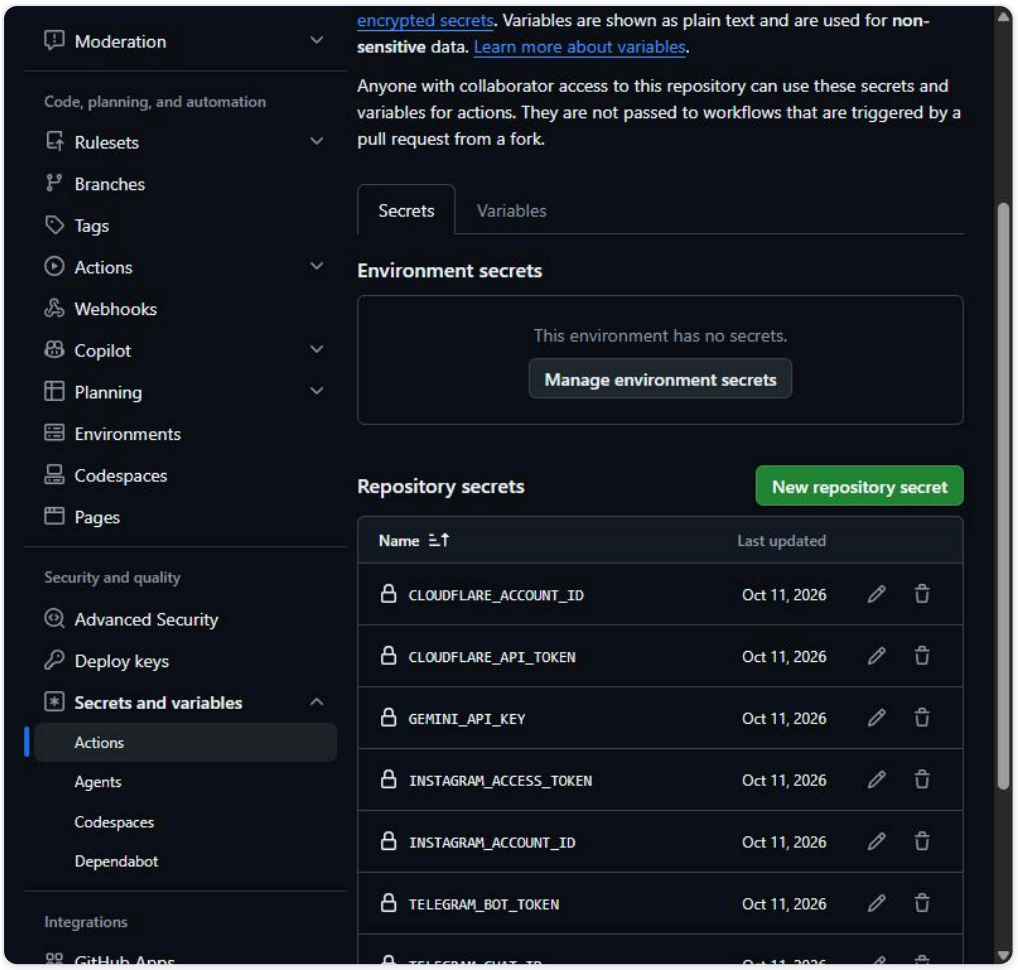


Figure 5. The secrets list. Only names and dates are visible; GitHub never shows the values again. (Screenshot shows names only — safe to look at, but it is still a good habit not to share it.)

> **CHECK**
> If a value has to be changed later you cannot "edit" it — click the pencil icon and paste the new value (*replace*).

8 Create the GitHub token for the Worker

≈ 5 min
yields value #8

The Worker needs permission to start workflows in your repository. Create a **fine-grained** token that can do *only* that.

1. GitHub → profile picture → **Settings → Developer settings → Personal access tokens → Fine-grained tokens → Generate new token**.
2. **Token name:** `ig-bot-relay`. **Expiration:** pick the longest offered and note the date (the maximum is often one year; 90 days were used in the original setup).
3. **Repository access:** *Only select repositories* → choose `ig-bot`.
4. **Permissions → Repository permissions → Contents → Read and write.** (Metadata: Read-only is added automatically.) That single permission allows `repository_dispatch`.
5. Click **Generate token**, confirm the summary, then **copy the token at once** — GitHub shows it only once. Save it as `GH_TOKEN`.

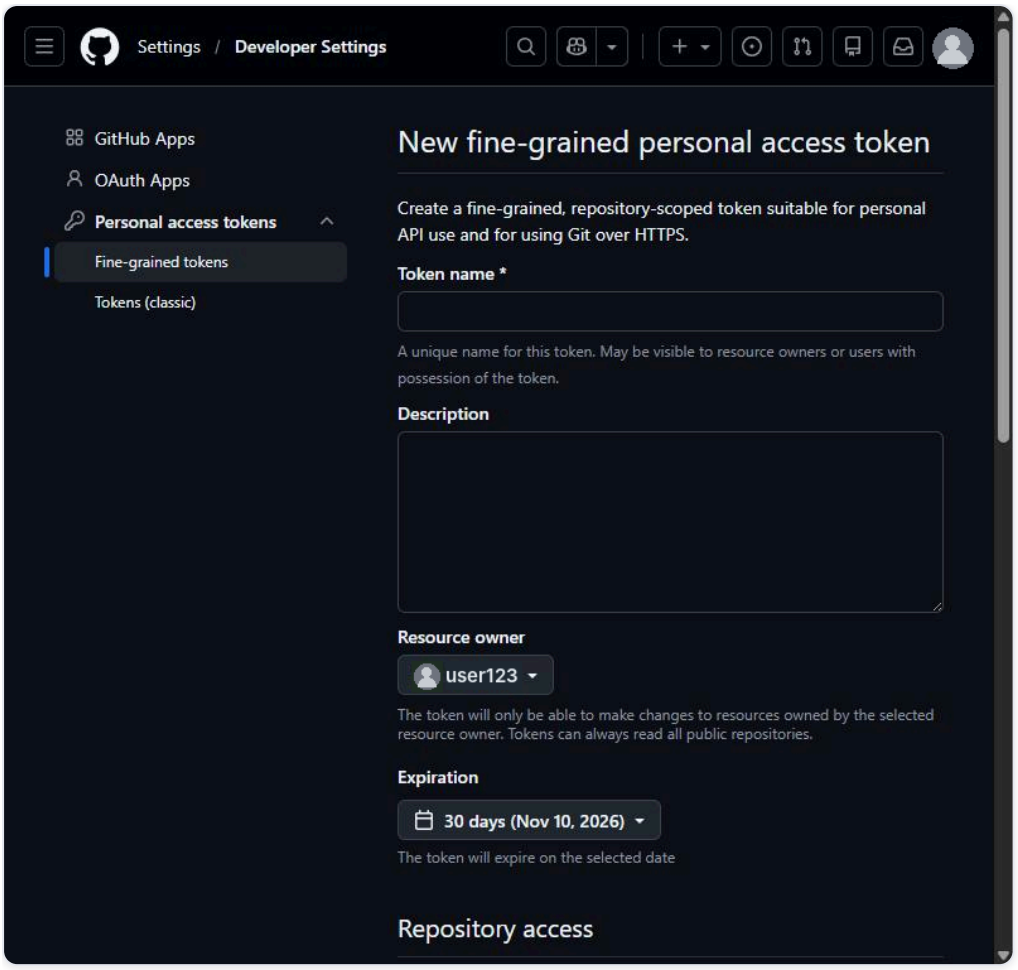


Figure 6. The token form (top part). Name, resource owner (your account) and expiration; below it you pick the repository and the permission.

> **DO NOT SCREENSHOT THE GENERATED TOKEN**
> The "Make sure to copy your personal access token now" screen shows the token in full. This guide intentionally has no picture of it.

> **WHY NOT A CLASSIC TOKEN?**
> Classic tokens grant access to *all* your repositories. A fine-grained token confined to one repository limits the damage if it ever leaks.

9 Deploy the Cloudflare Worker

≈ 10 min
yields value #9

9.1 Create the Worker

1. Cloudflare dashboard → **Workers & Pages → Create → Start with Hello World!**.
2. Name it `ig-bot-relay` and click **Deploy**. Your Worker URL will look like `https://ig-bot-relay.<your-subdomain>.workers.dev`.

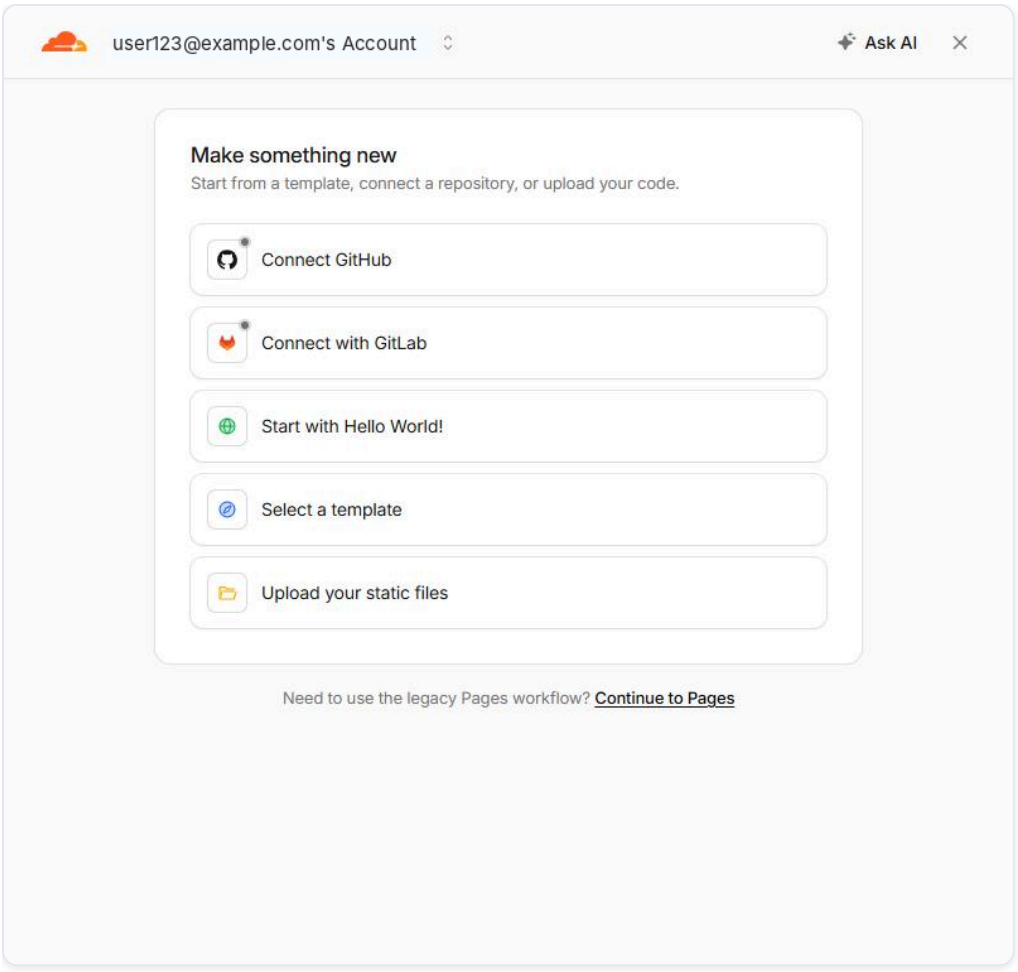


Figure 7. "Make something new" → **Start with Hello World!**. (Account name anonymised.)

9.2 Replace the code

1. Open the Worker → **Edit code**. Click into the code, press Ctrl + A (⌘ + A on Mac) and paste the content of `worker/worker.js` (also printed in the Appendix).
2. Click **Deploy**.

9.3 Add the variables and secrets

Worker → **Settings → Variables and Secrets → Add**. Add five entries; for the first four tick **Secret** (type "Secret") so the value is encrypted:

Name	Type	Value
`WEBHOOK_SECRET`	Secret	A random string you invent, **only letters, digits, `_` and `-`**, e.g. `igbot_ExampleSecret_ChangeMe42`. Write it down — you need the same string in Step 10.
`TELEGRAM_CHAT_ID`	Secret	Your chat ID from Step 3
`TELEGRAM_BOT_TOKEN`	Secret	Bot token from Step 3
`GH_TOKEN`	Secret	The GitHub token from Step 8
`GITHUB_REPO`	Text	`your-github-username/ig-bot`

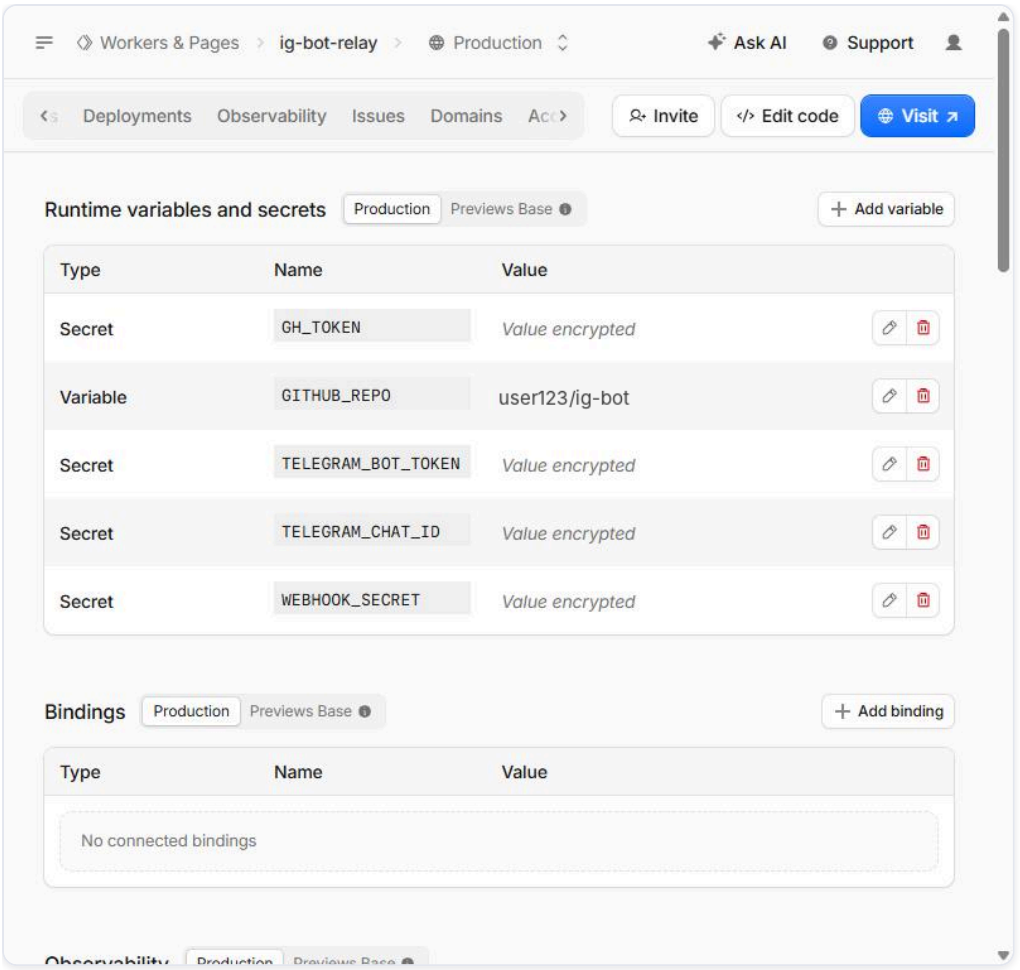


Figure 8. Worker settings after saving: secrets show "Value encrypted" and cannot be read back. The repo name is anonymised.

PASTING INTO THE EDITOR

If pasting the code does not work with Ctrl + V in the in-browser editor, check that the browser is allowed to access the clipboard (the editor lives in an embedded frame). Typing the code by hand is error-prone because the editor auto-closes brackets — prefer pasting.

10 Connect Telegram to the Worker

≈ 3 min

Tell Telegram to send every update to your Worker, with your secret header. Run **one** command in a terminal (this is the only time the secret and the token are typed in a terminal).

Windows (PowerShell)

```
curl.exe "https://api.telegram.org/bot123456789:AAH-your-token/setWebhook" -d "url=https://ig-bot-relay.YOUR-SUBDOMAIN.workers.dev" -d "secret_token=igbot_ExampleSecret_ChangeMe42"
```

macOS / Linux

```
curl "https://api.telegram.org/bot123456789:AAH-your-token/setWebhook" \
  -d "url=https://ig-bot-relay.YOUR-SUBDOMAIN.workers.dev" \
  -d "secret_token=igbot_ExampleSecret_ChangeMe42"
```

Replace the example token, subdomain and secret with **your** values — no angle brackets, no spaces. A correct answer is:

```
{"ok":true,"result":true,"description":"Webhook was set"}
```

If you get an error

Message	Meaning and fix
`404 Not Found`	The bot token in the URL is wrong. It must be written as `bot` immediately followed by the token (`bot123456789:AAH…`), without `< >` .
`secret token contains illegal characters`	The secret may contain only `A–Z a–z 0–9 _ -` . Make a new one and update `WEBHOOK_SECRET` in the Worker (Settings → the secret → **Rotate** → paste → Deploy), then repeat the command with the same string.
`bad webhook: HTTPS url must be provided`	The Worker URL is missing `https://` or has a typo.

Verify (optional): open `https://api.telegram.org/bot<TOKEN>/getWebhookInfo` — it should show your Worker URL, `pending_update_count` close to 0 and no `last_error_message` . Do not paste this output anywhere: it contains no token, but the URL you typed does.

> **THIS COMMAND CONTAINS YOUR BOT TOKEN AND SECRET**
> Do not copy it into chats, tickets or screenshots. Close the terminal when done; on Windows you may also clear the history with `Clear-History` .

11 First run and end-to-end test

≈ 10 min

1. **Test the relay:** send `/status` to your bot. Within 30–60 seconds it should reply "Pending drafts: 0 / Waiting for your feedback: no". This proves the full chain Telegram → Worker → GitHub → Telegram.
2. **Create the first draft:** GitHub → **Actions → Daily draft → Run workflow** (or send `/generate`). After 1–3 minutes a draft with images and two buttons arrives in Telegram. This also creates the `data` branch.
3. **Test Reject first (harmless):** tap **Reject / Edit**, reply with feedback such as "make it shorter and punchier" — you should receive a new version.
4. **Test Approve** only when you are happy for the post to go live: the bot answers "Publishing to Instagram..." and then "Published!" with a link.

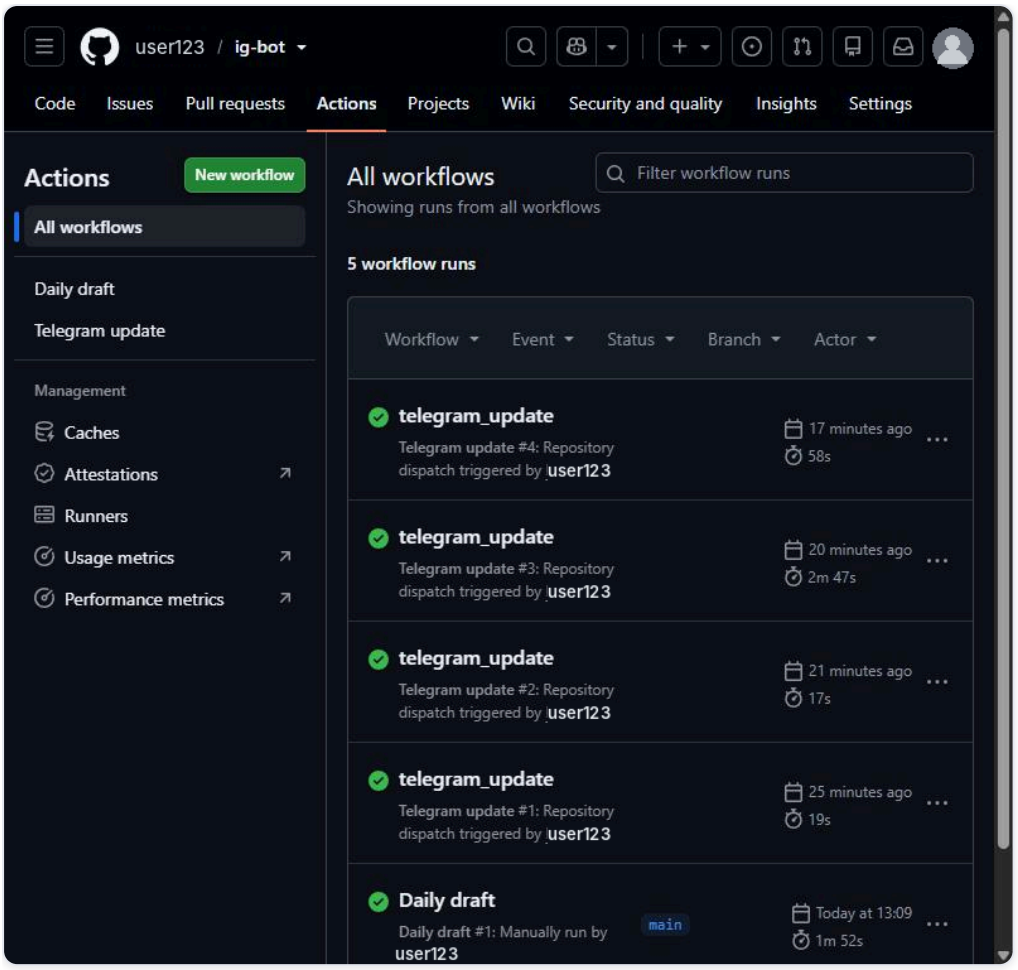


Figure 9. Actions tab: one **Daily draft** run (manual) and several **Telegram update** runs, each started by one of your messages or taps. A green check means the job finished without an error.

GREEN ≠ DELIVERED

The bot reports most errors *to you in Telegram* and finishes the job normally, so a green run does not always mean the post worked. Read the Telegram message; if there is none, open the run and read its log.

SHUTTING DOWN THE OLD SERVER

If you are migrating from the paid, always-on version of this bot, set the new webhook (Step 10) first, test, and only then delete or suspend the old hosting service so you stop being billed.

4 · Daily use & customisation

4.1 Your daily routine

1. A preview arrives in Telegram around 07:00 UTC (09:00 in summer in Central Europe, 08:00 in winter).
2. Read the caption and look at the slides. Tap **Approve & Publish** or **Reject / Edit** and reply with feedback.
3. Wait up to a minute for the confirmation. If nothing is approved, nothing is published.

4.2 Bot commands

Command	Effect
`/generate`	Create a new draft immediately (1–3 minutes).
`/status`	Shows how many drafts are pending and whether the bot is waiting for your feedback. It only reports — it does not send images.
`/cancel`	Stop waiting for feedback on a rejected draft.
`/start`	Shows the help text.

4.3 Customising without touching code

Set **Repository variables** (Settings → Secrets and variables → Actions → **Variables** tab). They are not secret. Anything you leave out uses the default from `config.py`.

Variable	What it changes
`BRAND_HANDLE`	Your Instagram handle printed on slides, e.g. `@yourname`
`BRAND_KICKER`	Small label on the slides (default "AI & TECH DAILY")
`BRAND_ACCENT`	Accent colour, hex (default `#8B5CF6`)
`IMAGE_STYLE`	Text prompt that sets the look of generated backgrounds
`IMAGE_PROVIDER`	Which image generator to use (Cloudflare FLUX is the free default when its secrets exist)
`RSS_FEEDS`	Comma-separated list of feed URLs — the bot's "news diet"
`MAX_SLIDE_IMAGES`	Upper limit of slides per carousel
`GEMINI_TEXT_MODEL` / `GEMINI_FALLBACK_MODEL`	Primary model and an optional second one used if the first fails or is rate-limited
`INSTAGRAM_GRAPH_VERSION`	Graph API version string, e.g. `v22.0`

4.4 Changing the posting time

Edit `.github/workflows/daily.yml` (pencil icon on GitHub) and change the cron line, e.g. `cron: "30 5 * * *"` = 05:30 UTC daily. Remember it is **UTC**; GitHub may start the job several minutes late, and during busy periods occasionally skips a run — that is why the manual **Run workflow** and `/generate` exist.

4.5 Behaviour to be aware of

- **Button latency:** 20–60 seconds, because each tap boots a fresh runner. Do not tap repeatedly; one tap is enough.
- **One job at a time:** several rapid taps queue up; GitHub keeps only one waiting job per group, so an excess tap may be dropped. Tap again if nothing happens after a minute.
- **Old messages:** buttons on drafts that are no longer pending answer "This draft is no longer active."
- **Inactivity pause:** GitHub may disable scheduled workflows in a public repository after about 60 days without repository activity. The daily workflow tries to re-enable itself on each run; if the daily draft ever stops, go to Actions → Daily draft → **Enable workflow**.

5 · Keeping it running

5.1 Key inventory — what expires and when

Almost everything that can fail in this system fails because a credential expired or was replaced. Keep this table (with your real dates) in your notes and set **calendar reminders**.

Credential	Lifetime	Stored in	What happens on expiry · how to renew
`INSTAGRAM_ACCESS_TOKEN`	≈ 60 days (long-lived user token)	GitHub secret	Publishing fails ("Error validating access token"). Repeat Step 6.3–6.4 *before* day 60 and replace the secret. Reminder every ~45 days.
`GH_TOKEN` (fine-grained)	As chosen, max about 1 year	Worker secret	Buttons stop working (Telegram taps are received but no workflow starts). Create a new token (Step 8), then Worker → Settings → `GH_TOKEN` → **Rotate** → paste → Deploy. GitHub e-mails a warning shortly before expiry.
`GEMINI_API_KEY`	No fixed expiry	GitHub secret	Fails only if deleted/blocked or limits are reached. Rotate if leaked: create a new key in AI Studio, replace the secret, delete the old key.
`CLOUDFLARE_API_TOKEN`	No fixed expiry (unless you set one)	GitHub secret	Image generation fails. Roll or recreate the token in Cloudflare, replace the secret.
`TELEGRAM_BOT_TOKEN`	No expiry	GitHub secret + Worker secret	Replace in **both** places if you regenerate it via BotFather (`/revoke`), then re-run `setWebhook` with the new token.
`WEBHOOK_SECRET`	No expiry	Worker secret (and your setWebhook command)	If changed, run `setWebhook` again with the identical new value or every update is rejected with 403.
Meta App Secret	No expiry	Meta dashboard only	Used only to mint tokens. Reset it if exposed.

THREE REMINDERS IS ALL YOU NEED

Add three recurring calendar entries: **every 45 days** – renew the Instagram token; **2 weeks before the GitHub token's expiry date** – replace `GH_TOKEN` ; **every 3 months** – glance at Actions for failed runs and at the free-tier dashboards.

5.2 How to rotate a secret safely (generic procedure)

1. Create the **new** credential at the provider (do not delete the old one yet).

2. Replace the value where it is stored (GitHub: Settings → Secrets → pencil; Cloudflare: Worker → Settings → secret → Rotate; then **Deploy**).
3. Test: `/status` for the Worker chain, `/generate` for Gemini/Cloudflare, an Approve for Instagram.
4. Only now revoke/delete the old credential at the provider.

5.3 If a secret leaks

- Revoke it at the provider **first** (delete the GitHub token / roll the API key / `/revoke` in BotFather / reset the Meta app secret), then do the rotation procedure.
- Leaked keys in a **public** repository are found by automated scanners within minutes. Deleting the file afterwards is not enough — the commit history keeps it. Always revoke.
- Never upload your real `.env` file. The included `.gitignore` excludes it, but drag-and-drop uploads in the browser ignore `.gitignore` — double-check what you select.

5.4 Security hygiene checklist

- Turn on **two-factor authentication** for GitHub, Cloudflare, Google, Meta and Telegram. The bot controls a public Instagram account; whoever owns your GitHub account owns the bot.
- Keep keys in a password manager, not in screenshots, chat messages or e-mails.
- Give every key the minimum scope (one repository, one permission).
- Do not make drafts about confidential topics: the `data` branch is publicly readable until publication.
- Do not accept pull requests or add collaborators to the repository unless you know them; workflow secrets are not exposed to forks, but keep it that way.

5.5 Monitoring the system

- **Telegram is your dashboard:** every failure to generate or publish is reported there.
- **GitHub Actions** (Actions tab): a red cross means the job crashed; click the run → the failing step to read the log. Secrets are masked in logs.
- **Cloudflare Worker logs:** Worker → **Observability** (or Logs) shows incoming requests and the line "GitHub dispatch failed <status>" if the relay could not reach GitHub (401 = token expired/wrong, 404 = wrong `GITHUB_REPO` or missing permission).
- **Telegram webhook health:** `getWebhookInfo` shows `last_error_message` and `pending_update_count`.
- **Provider dashboards:** Workers AI usage in Cloudflare, quota in Google AI Studio.

5.6 Updating the bot's code

Edit files directly on GitHub (pencil icon) or upload a replacement file with **Add file → Upload files**. Changes apply to the next run. Keep a copy of the original `free-version` folder so you can undo a change. Changing `worker.js` additionally requires pasting the new code into the Worker and clicking Deploy.

5.7 Costs — and how to keep them at zero

- Keep the repository **public** (a private one is limited to 2,000 Actions minutes per month and Instagram could not read the images anyway).
- Do not add a payment method to Cloudflare or Google unless you intend to — without one, hitting a limit simply stops the feature instead of billing you.
- If you ever want to stop everything: disable the workflows (Actions → workflow → ⋯ → Disable), delete the Worker, and call `deleteWebhook` on the Telegram Bot API.

6 · Troubleshooting

Symptom	Likely cause	What to do
No reply to `/status`	Webhook not set, wrong secret, or Worker code/variables missing	Run `getWebhookInfo`. If `last_error_message` says 403 → the `WEBHOOK_SECRET` in the Worker differs from `secret_token` in setWebhook. If the URL is empty → run Step 10 again.
`/status` works for others but not for you	Wrong `TELEGRAM_CHAT_ID`	The Worker silently ignores unknown chats. Re-check the ID (digits only) in both GitHub and the Worker.
Tap on a button: nothing happens	Expired/invalid `GH_TOKEN`, wrong `GITHUB_REPO`, or the token lacks *Contents: write*	Check Worker logs for "GitHub dispatch failed 401/403/404". Create a new token (Step 8) and rotate the secret.
Actions run is green but no draft arrives	An error was caught and reported — or Telegram secrets are wrong	Open the run log. A "⚠️ Draft generation failed" message in Telegram names the cause (quota, API key, etc.).
Draft arrives, **Approve** fails with a Meta error	Expired token, wrong account ID, missing permission, or Instagram cannot download the image	Test the token with the URL in 6.6. Open one image URL (`raw.githubusercontent.com/<user>/ig-bot/data/media/<id>/slide_1.jpg`) in a private window: it must display without logging in (repository must be public, `data` branch must exist).
Image generation error	Daily free neurons used up, or wrong Cloudflare token/ID	Wait for 00:00 UTC reset, or check the two Cloudflare secrets.
"Gemini … 429 / quota"	Per-minute or per-day free limit	Wait and retry; set `GEMINI_FALLBACK_MODEL`; reduce regenerations.
Daily draft did not come	GitHub skipped/disabled the schedule	Actions → Daily draft → Enable workflow, then **Run workflow**. Check that `daily.yml` is in `.github/workflows`.
`404` from setWebhook	Token typed with `< >` or missing `bot` prefix	See Step 10.
Two answers / double publish concern	—	Not possible by design: the button keyboard is removed after one tap and the draft status is saved as *published* immediately. A failed publish re-sends the buttons so you can retry.

Appendix · Cheat-sheet and Worker code

A. Where everything lives

Thing	Location
Source code, workflows	GitHub repo `your-user/ig-bot`, branch `main`
Draft state & images	Branch `data` of the same repo
Daily schedule	`.github/workflows/daily.yml` (cron, UTC)
Telegram event handler	`.github/workflows/telegram.yml`
Secrets for the bot	GitHub → Settings → Secrets and variables → Actions
Relay secrets	Cloudflare → Workers & Pages → ig-bot-relay → Settings → Variables and Secrets
Manual draft	Actions → Daily draft → Run workflow, or `/generate`

B. Useful URLs (replace the placeholders)

```
# webhook status
https://api.telegram.org/bot<TOKEN>/getWebhookInfo
# remove the webhook (stops all button handling)
https://api.telegram.org/bot<TOKEN>/deleteWebhook
# Instagram token test
https://graph.facebook.com/v22.0/<IG_ID>?fields=username&access_token=<TOKEN>
```

C. `worker/worker.js`

```
export default {
  async fetch(request, env, ctx) {
    if (request.method !== "POST") return new Response("ok");

    if (request.headers.get("X-Telegram-Bot-Api-Secret-Token") !== env.WEBHOOK_SECRET) {
      return new Response("forbidden", { status: 403 });
    }

    let update;
    try { update = await request.json(); }
    catch { return new Response("bad request", { status: 400 }); }

    const chatId = update.message?.chat?.id ?? update.callback_query?.message?.chat?.id;
    if (String(chatId) !== String(env.TELEGRAM_CHAT_ID)) return new Response("ok");

    if (update.callback_query) {

ctx.waitUntil(fetch(`https://api.telegram.org/bot${env.TELEGRAM_BOT_TOKEN}/answerCallbackQuery`, {
        method: "POST", headers: { "content-type": "application/json" },
        body: JSON.stringify({ callback_query_id: update.callback_query.id, text: "Working on it…"
}),
      }));
    }
    ctx.waitUntil(dispatch(update, env));
    return new Response("ok");
  },
};

async function dispatch(update, env) {
  const resp = await fetch(`https://api.github.com/repos/${env.GITHUB_REPO}/dispatches`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${env.GH_TOKEN}`,
      Accept: "application/vnd.github+json",
      "X-GitHub-Api-Version": "2022-11-28",
      "User-Agent": "ig-bot-relay",
      "content-type": "application/json",
    },
    body: JSON.stringify({ event_type: "telegram_update",
      client_payload: { update: JSON.stringify(update) } }),
  });
  if (!resp.ok) console.error("GitHub dispatch failed", resp.status, await resp.text());
}
```

D. Glossary

RSS	A standard feed format in which websites list their newest articles.
API key / token	A secret string that proves to a service that a request comes from you.
Webhook	A URL that a service calls when an event happens.
Neuron	Cloudflare's billing unit for Workers AI usage.
FLUX	The open image-generation model family used for backgrounds (the "schnell" variant is the fast one).

Pillow	The Python imaging library that draws text and layouts onto the generated pictures.
Carousel	An Instagram post with several swipeable images.
Force-push	Overwriting a Git branch with a new history; used so the `data` branch always holds exactly one commit.

Document version: October 2026. Service dashboards, limits and API versions change; where this guide and a provider's current documentation disagree, follow the provider.